

Are LLMs Reliable for Recommending Libraries and Their Usage? An Empirical Study

Anonymous Author(s)*

ABSTRACT

Large Language Models (LLMs) are increasingly used by developers as alternatives to traditional question-and-answer platforms for programming assistance, particularly for library recommendation and code usage guidance. Despite their widespread adoption, the reliability of LLMs in this context remains insufficiently understood, especially given the known risks of hallucination and security vulnerabilities in software dependencies.

In this paper, we conduct an empirical study to evaluate the reliability and security awareness of LLM-based library recommendations. We investigate two key aspects: (1) the quality of recommended library versions and their associated usage patterns embedded in library choices, and (2) the extent to which LLMs can recognize and mitigate security risks using external Skills. To this end, we design a systematic evaluation across multiple real-world Stack Overflow scenarios involving popular Java libraries and assess several mainstream LLMs. We find that while LLM outputs are often fluent and usable, they frequently suffer from version and code insecure recommendations. Moreover, even when explicitly prompted with security considerations, LLMs only partially improve their recommendations and remain prone to unsafe or incompatible usage suggestions. We further observe that adapting code to secure library versions remains challenging even with explicit migration guidance, indicating limited robustness in LLM-based software maintenance tasks. Overall, our study highlights important limitations in the reliability and security awareness of LLMs and underscores the need for future safeguards.

1 INTRODUCTION

Large language models (LLMs) are reshaping how developers seek programming knowledge and assistance, increasingly replacing traditional question-and-answer (Q&A) platforms. Instead of searching and browsing community-curated answers, developers now rely on LLMs to obtain instant, context-aware responses and code suggestions. In contrast, the usage of platforms such as Stack Overflow has experienced an unprecedented decline [36] in recent years.

Among the various ways developers rely on LLMs, library recommendation [4, 15] constitutes as one of the most common and practical use cases. Modern software development heavily depends on third-party libraries. Developers rely on library recommendation to identify appropriate libraries, select suitable versions, and obtain correct usage examples. As developers increasingly turn to LLMs, the reliability of their recommendations for libraries, versions, and usage examples has become a critical concern. Unfortunately, developers blindly expect LLMs to provide concise and actionable recommendations tailored to their specific contexts, and tend to rely on these results with a high level of trust.

However, this growing trust in LLMs is not always justified. Despite their impressive capabilities, LLMs are known to produce hallucinations [28, 41, 49]. The LLMs may recommend non-existent

or incorrect library versions. Such issues are particularly concerning in practice, as software libraries often contain known security vulnerabilities (e.g., CVEs) [22], and incorrect version suggestions may expose systems to serious security risks. Beyond recommending library versions, LLM-generated code also suffers from hallucination and quality issues, raising critical concerns about the reliability of LLMs in supporting library recommendations.

Existing studies evaluate the reliability of LLM-generated content from diverse perspectives. For general Q&As, recent research shows that ChatGPT answers contain incorrect and verbose information, despite the fact that users preferred ChatGPT answers and overlooked the misinformation frequently [21]. Another study on stack overflow posts shows that while LLMs can partially substitute for human expertise, they still fall short of outperforming human experts in certain domains [5]. For the security aspect, recent studies investigate the prevalence of vulnerable code by GitHub Copilot [37], and reveals that LLMs often overlook security considerations when answering programming-related queries [38]. Their work focuses on the security of LLM-generated code. However, it does not assess the reliability of LLMs when recommending library versions and their associated code usage. There is also work on using LLMs for library migration [18]. However, it does not evaluate their ability to adapt code usages from vulnerable to secure library versions. As a result, there lacks a systematic study of the reliability and security awareness of LLM-based library recommendations, leaving a critical gap that is increasingly overlooked as LLM usage grows.

In this paper, we aim to bridge this gap by conducting an empirical study on the reliability of LLMs in recommending libraries, and their usages, as well as the security awareness embedded in the LLMs. Our study is motivated by two practical concerns. First, LLMs may recommend incorrect or hallucinated library versions, hindering the development process. Second, they may generate code usage that is incompatible with the recommended version or introduces security vulnerabilities, thereby undermining secure development. To this end, we perform the **Version Recommendation Quality (RQ1)** and **Code Recommendation Quality (RQ2)** to evaluate whether LLMs, as condensed representations of vast knowledge bases, can recognize the security implications of different library versions and self-correct their recommendations. Furthermore, we conduct **Security-aware Skill Effectiveness (RQ3)** and **Security-aware Migration Skill Effectiveness (RQ4)** to observe whether using external prompting mechanisms (Skills) can help elicit LLMs' awareness of security considerations.

We construct a dataset from 7,887 relevant Stack Overflow posts, covering 21 popular Java libraries, and evaluate 3 mainstream LLMs. Our study reveals several noteworthy findings. *First*, LLM answers do not necessarily imply reliable version-level decisions: under a standard usage scenario, DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview recommend vulnerable library versions in 44.50%,

25.08%, and 38.87% of cases, respectively; adding a lightweight security-aware Skill reduces these rates to 13.94%, 8.04%, and 2.68%. *Second*, when explicitly provided with a safe library version, correct migration remains difficult: only 3.6%, 5.4%, and 19.1% of code-generation cases complete migration successfully for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively, while many answers still use wrong artifacts, omit required companion dependencies, or retain outdated API assumptions. *Third*, lightweight migration Skill yields only partial improvements, suggesting that safe version adaptation remains challenging. These observations indicate that current LLMs can produce usable answers, but still lack reliable built-in security awareness and robust safe-version consistent migration capability for library recommendation tasks.

In summary, this paper makes the following contributions:

- We construct a dataset from 7887 Stack Overflow posts covering 21 popular Java libraries for library version and code recommendation.
- We evaluate the performance of LLMs in both library version and code recommendation, revealing a substantial gap of their practical usefulness and associated risks.
- We observe the effectiveness of lightweight security-aware Skills for both version recommendation and code migration.

2 RELATED WORK

2.1 LLM for Recommendation

Recent advances in LLMs have enabled their application as powerful recommenders. A prominent research direction focuses on enhancing recommendation quality. Lin et al. [26] token-level collaborative alignment that aligns generated tokens with collaborative signals to better capture fine-grained user preferences. Wang et al. [45] investigate the problem of context bias in LLM-based recommenders, showing that models may focus on irrelevant words and proposing mitigation techniques. To enhance reasoning capability, Li et al. [23] introduce ScotRec, which incorporates social-aware chain-of-thought reasoning into recommendation generation. Liu et al. [30] propose *LLM-ESR* which enhances long-tailed sequential recommendation by leveraging the semantic understanding of LLMs. Wei et al. [46] present *LLMRec*, a framework that augments LLMs with graph-structured user-item interaction data to improve recommendation accuracy. Han et al. [13] rethink LLM-based recommendation by proposing a personalized query-driven parallel integration framework. To systematically evaluate LLMs as recommenders, Jiang et al. [19] argue that evaluation should go beyond traditional utility metrics, emphasizing aspects such as explainability and user satisfaction. Huang et al. [17] propose a benchmark to provide standardized evaluation settings for personalized recommendation scenarios.

In the software engineering domain, LLM-based recommendation has been applied to package and library recommendation tasks. He et al. [15] introduce MigrationAdvisor, which recommends library migrations using large-scale open-source data. Liu et al. [29] develop Legommenders, a comprehensive recommendation library enhanced with LLM capabilities for content-based recommendation. Cui et al. [4] conduct a large-scale empirical study on recommending open-source software, highlighting practical challenges

in real-world ecosystems. However, emerging evidence also reveals limitations of LLM-based recommenders. Spracklen et al. [41] demonstrate that LLMs may hallucinate non-existent software packages. Sajadi et al. [38] show that LLMs often overlook security considerations when answering programming-related queries. Da Silva et al. [5] and Kabir et al. [21] analyze the reliability of LLM-generated answers compared to Stack Overflow, highlighting issues in trustworthiness and practical usability.

2.2 LLM for Code Generation

LLMs have become a major paradigm for automated code generation. One important line of research investigates how testing can be integrated into LLM-based code generation. Mathews et al. [31] explore the combination of test-driven development and LLM-based code generation to produce functionally correct implementations. Bose [1] proposes a property-based testing framework that shifts the focus from prompt engineering to formal behavioral properties. Another major direction evaluates the quality and performance of LLM-generated code. Li et al. [24, 25] conduct an empirical case study on GitHub Copilot, and compare multiple state-of-the-art models to provide a broader understanding of performance differences across tools. For algorithmic programming tasks, Coignon et al. [3] analyze the performance of LLM-generated solutions on LeetCode problems, while Zhang et al. [48] investigate stylistic variations in generated code for the same benchmark. Beyond general-purpose languages, Erkuş et al. [9] provide a comprehensive evaluation of LLMs for ArkTS code generation. Furthermore, Joel et al. [20] survey the application of LLMs to low-resource and domain-specific programming languages.

To support systematic assessment, Du et al. [7] introduce CodeArena, a collective evaluation platform specifically designed for LLM-based code generation systems. Similarly, Shah et al. [40] propose StackEval a benchmark for evaluating coding assistance capabilities of LLMs in practical programming scenarios. Santa Molison et al. [39] compare LLM-generated code with human-written code in terms of maintainability and reliability, offering insights into long-term software quality. Recent studies reveal several risks and limitations of LLM-based code generation. Hallucination is a critical concern [28, 49]. In addition, with regard to fairness and bias, Huang et al. [16] investigate bias testing and mitigation techniques, while Ling et al. [27] uncover social bias embedded in generated code outputs. Pearce et al. [37] assess the security implications of Copilot's code suggestions and identify vulnerabilities.

2.3 LLM for Migration

Early software migration studies primarily focus on understanding migration practices and challenges through empirical analysis. Teyton et al. [44] conduct one of the first systematic studies on Java library migrations, analyzing common patterns and motivations. Kula et al. [22] further investigate whether developers update dependencies in response to security advisories, revealing that migration decisions are often influenced by security concerns but remain inconsistent in practice. He et al. [14] provide a large-scale empirical study on Java library migrations, examining their prevalence, trends, and underlying rationales across open-source projects. Building on these foundations, recent work explores the

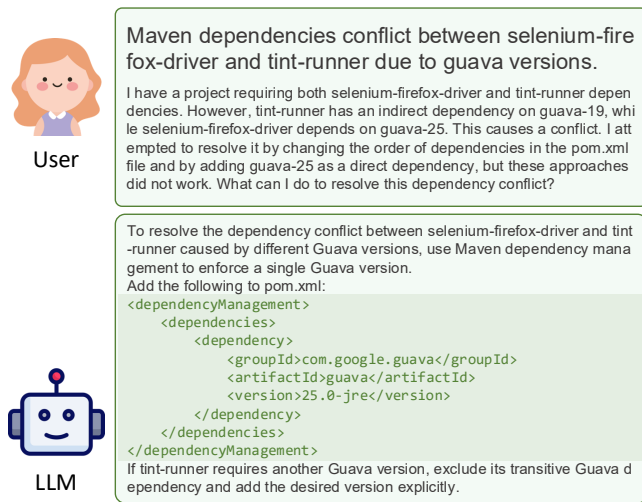


Figure 1: Illustrative Example of Using an LLM to Recommend the Correct Library Version for a Guava Version Conflict

use of LLMs to automate and improve migration tasks. Zhou et al. [52] propose a hybrid API migration approach that combines small API mapping models with LLMs, leveraging the strengths of both structured mappings and generative capabilities. Islam et al. [18] conduct an empirical study on using LLMs for library migration, demonstrating that LLMs can assist in generating migration code but still face challenges in correctness and completeness. In addition to general-purpose migration, domain-specific migration scenarios have received increasing attention. Zhang et al. [47] introduce Cross2OH, a system designed to support the porting of C/C++ libraries to the OpenHarmony platform. Similarly, Zhou et al. [51] study the migration of software libraries from TypeScript/JavaScript to ArkTS. To facilitate systematic evaluation of LLM-based migration techniques, Cheng et al. [2] propose CodeMeny, a benchmark specifically designed to evaluate LLMs on code migration tasks, providing standardized datasets and evaluation protocols for comparing different approaches.

3 EMPIRICAL STUDY

Figure 3 presents the overall process of our study. In the study setup phase, we collect libraries and versions from repositories, gather relevant Stack Overflow posts, perform data rephrasing and classification, and conduct result annotation and LLM-based recommendation generation. Next, in the *recommendation quality evaluation* phase, we evaluate both version and code recommendation quality (RQ1 and RQ2, resp.) from perspectives including basic quality and security-sensitive quality. Finally, in the skill effectiveness phase, we assess the effectiveness of security-aware capabilities, including library recommendation (RQ3) and migration skills (RQ4).

3.1 Illustration Examples

Recommending a Library Version (Figure 1). In various scenarios, users may rely on an LLM to recommend an appropriate library and version to accomplish their tasks. In this scenario,

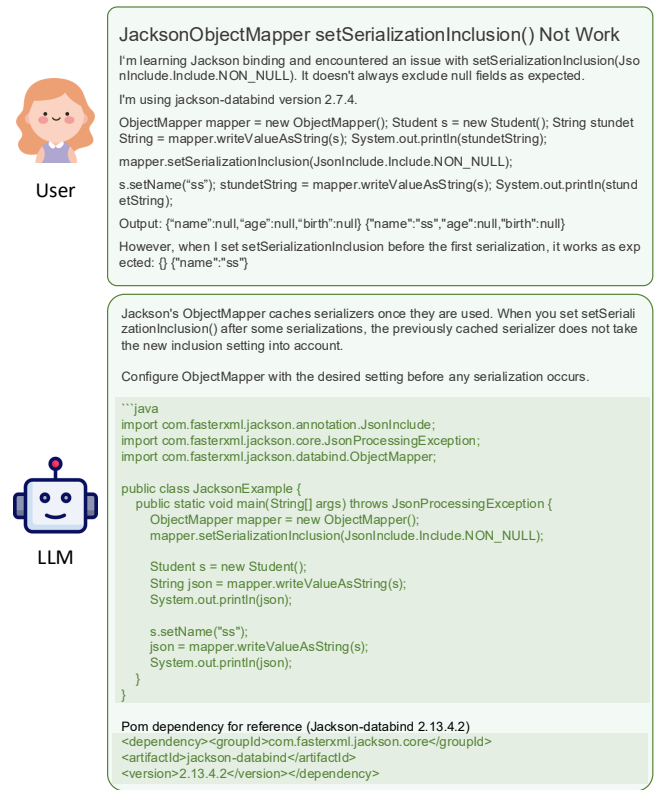


Figure 2: Illustrative Example of Using an LLM to Recommend Library Usage under a Safe Jackson Databind Version

the user confronts a Maven dependency conflict caused by different Guava versions introduced by selenium-firefox-driver and tint-runner [43]. An LLM might be leveraged to identify the conflict problem and provide a concrete dependency resolution. i.e., a recommended `<groupId>`, `<artifactId>` and `<version>` (GAV) of `com.google.guava`, `guava`, `25.0-jre`. However, the LLM response simply follows the Guava version `25.0-jre` mentioned in the question, which is known to be vulnerable, and thus fails to provide a safe recommendation.

Recommending Library Usage under a Version (Figure 2).

In addition to version recommendation, users may ask an LLM to recommend library usage under an existing dependency context. In this scenario, the user uses `jackson-databind` version `2.7.4` and encounters unexpected behavior of `ObjectMapper`'s `setSerializationInclusion(JsonInclude.Include.NON_NULL)` API, where null fields are still serialized [8]. An LLM might be leveraged to analyze the issue and provide a concrete solution under to a safe version. In our previous attempts, our target LLM suggests a code snippet with the `groupId`, `artifactId`, and version of `com.fasterxml.jackson.core`, `jackson-databind`, `2.13.4.2`, resp. However, the response omits required companion Jackson dependencies, so the safe-version migration is still incomplete.

3.2 Research Questions

We present the following four research questions:

- **Recommendation Quality:**

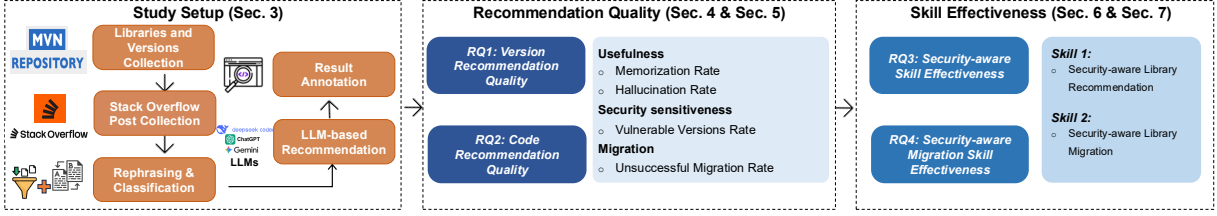


Figure 3: Overview of Our Empirical Study

- **Version Recommendation Quality (RQ1):** How is the quality of LLM-recommended library versions?
- **Code Recommendation Quality (RQ2):** How is the quality of LLM-recommended library usage based on recommended library versions?
- **Skill Effectiveness:**
 - **Security-aware Skill Effectiveness (RQ3):** How is the effectiveness of LLMs in recommending library versions using security-aware skills?
 - **Security-aware Migration Skill Effectiveness (RQ4):** How is the effectiveness of LLMs in mitigating insecure library usage using security-aware migration skills?

Overall, **RQ1** and **RQ2** characterize the quality of LLMs’ original recommendations while **RQ3** and **RQ4** further examine whether lightweight security-aware skills can reduce insecure recommendations and provide effective migration.

3.3 Dataset Construction

We focus on Java libraries, as the Java ecosystem offers a mature dependency management infrastructure (e.g., Maven), along with an intensive volume of real-world development and discussions. The construction process comprises libraries and versions collection, stack overflow post collection, data rewritten and data classification.

3.3.1 Libraries and Versions Collection. We first collect popular Java libraries as the target libraries of our study. We obtain 65 candidate libraries from the Maven repository, where it provides the ranking of popular artifacts [32]. We exclude artifacts outside general Java library usage, such as Kotlin, Scala, Clojure, Renjin/R, and Android/Kotlin extension artifacts. The final set contains 53 Java libraries, covering categories such as testing, logging, utilities, JSON processing, web development, HTTP clients, and databases. Formally, each Java library version is represented by its Maven coordinate, i.e., $l = \langle g, a, v \rangle$, where g , a and v denote the groupId, artifactId, version, respectively. The selected library set is denoted as $\mathcal{L}$ Based on the 53 Java libraries.

Meanwhile, we construct a vulnerable library database. We collect vulnerability records from public sources, including NVD [33], GitHub Advisory Database [10], and OSV [34] before August 2024. We manually parse the vulnerability records by their affected version ranges and represent each vulnerability record as $\langle g, a, \mathcal{R}_{vul} \rangle$, where $\mathcal{R}_{vul}$ denotes the set of vulnerable version ranges. To retain Java-related entries, we keep only records associated with the Maven ecosystem, yielding 10,491 vulnerability records. We then match these records against 53 popular Java libraries and check whether their released versions fall within the affected version ranges. As a result, we obtain 176 vulnerability records covering 21 target Java libraries, denoted as $\mathcal{VDB}$.

Rewriting prompt template:

Task: Rewrite the Stack Overflow question, while preserving its original technical meaning.

Input: The original question title and body.

Preservation rule:

Keep all target-library-related information verbatim, including library names, group IDs, artifact IDs, versions, APIs, code snippets, configurations, and error messages.

Constraints:

Do not add or remove technical elements, introduce back-ground information, or provide a solution.

Output: Return the rewritten_title and rewritten_body.

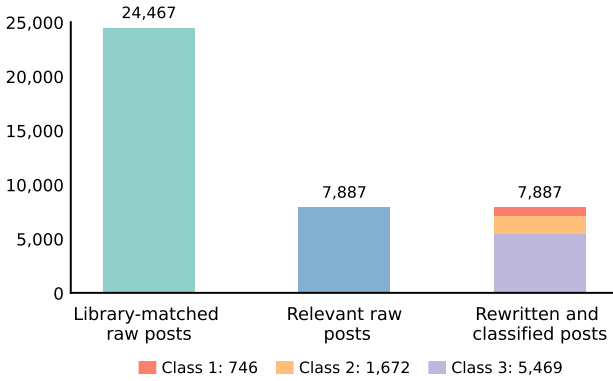
Figure 4: Prompt template for Rephrasing.

3.3.2 Post Collection. We use Stack Overflow [42] as the primary source for collecting posts in which developers request recommendations for Java libraries. We leverage the Google BigQuery Stack Overflow dataset [12], covering posts up to December 2022. Each Stack Overflow post with the accepted answer contains a title, question body and the answer content. To retrieve posts related to Java ecosystem and libraries, we search posts with accepted answers, tagged with java and match library names in the post title or body based on our selected 21 Java libraries.

For each post whose title or body matches a selected library name, we obtain a library-post tuple $\langle p, l \rangle$, where p is the matched Stack Overflow post and l is the matched target library. This retrieval process yields 245,086 candidate posts. Subsequently, we randomly sample 24,467 (10%) posts for further analysis, with a sample confidence of 99% and confidence interval of approximately 0.78. To precisely identify library-related posts and reduce false matches, we use DeepSeek-R1 [6] as an LLM judge [40, 50] to identify post relevance. This step reduces the raw set from 24,467 sampled library-name-matched posts to 7,887 relevant posts.

3.3.3 Rephrasing and Classification. Rephrasing. To reduce the possibility that LLMs reproduce memorized Stack Overflow answers, we use Deepseek-R1 [6] to rephrase each post before querying the models. The rephrase only changes the surface expression of the question while preserving its semantic intent. In particular, all library-related identifiers and technical evidence, such as library names, Maven coordinates, versions, APIs, code snippets, configuration items, and error messages, were required to remain unchanged. The rephrasing prompt is summarized in Figure 4.

Classification. The post classification identifies both the question scenarios and the corresponding answer content. We used DeepSeek-R1 [6] to categorize the rephrased posts based on the role of the target library in each question-answer pair. The classification labels class 1, class 2 and class 3, and the prompt is summarized

Classification prompt template:**Task:** Classify the Stack Overflow post with respect to the target library.**Input:** The question, accepted answer, and target library.**Label definitions:****Class 1** The question mentions the target library, and the answer provides its Maven GAV or a GAV that transitively introduces it.**Class 2** The question already provides such a GAV, and the answer contains target-library-related usage content.**Class 3** All other cases.**Output:** Return the class and reason.**Figure 5: Prompt template for post classification.****Figure 6: Distribution of collected and rephrased posts.**

in Figure 5. To reduce labeling noise, we manually inspected all retained Class 1 and Class 2 posts and a sample of Class 3 posts.

- **Class 1 (Version Recommendation task for RQ1 and RQ3):** The user asks about a specific library, and the answer recommends a particular groupId g , artifactId a , and version v .
- **Class 2 (Code Recommendation tasks for RQ2 and RQ4):** The user asks about a specific issue involving a library identified by its groupId, artifactId, and version, and the accepted answer provides code snippets demonstrating its usage. The invoked APIs in these snippets are grouped as $\mathbb{A}$.
- **Class 3 (Others).** Posts that do not satisfy Class 1 or 2 criteria.

In total, 746 posts are categorized as Class 1, 1,672 as Class 2, and 5,469 as Class 3. Figure 6 summarizes the distribution of the constructed dataset.

3.4 LLM Recommendation

We prompt multiple LLMs to generate answers for the research questions. We select three representative LLMs in our study, including **DeepSeek-R1** [6], **GPT-4o** [35], and **Gemini-3.0-Flash-Preview** [11]. These models are chosen to cover different model families and providers, enabling a comparative analysis under a unified evaluation setting. All models are accessed via their official APIs with default configurations unless otherwise specified.

Recommendation quality. The prompt templates for the **Class 1** and **Class 2** are illustrated in Figure 7 and Figure 8, respectively. For **Class 1**, the prompt requires the LLM to answer the question

Prompt Template for Version Recommendation (Class 1):**Input:**

- Matched Library: {target_lib}
- Question Title: {title}
- Question Body: {body}

Task:

- Answer the Stack Overflow question and provide a working solution.
- Provide a concise and accurate answer.
- Include reproducible dependency information for the target library.
- Select a specific version and use it consistently in the answer.

Output:

```
{
  "answer_body": "...",
  "target_lib_dependency": {
    "groupId": "...",
    "artifactId": "...",
    "version": "..."}
}
```

Figure 7: Prompt template for Class 1**Prompt Template for Safe-Version Code Migration (Class 2)****Input:**

- Matched Library: {target_lib}
- Safe Version: {safe_version}
- Stack Overflow Question: {title}+{body}

Task:

- Answer the question under the constraint of the given safe library version.
- Adapt the solution to the specified safe version.
- Provide both explanation and implementation code.
- Ensure consistency between description and code.

Output:

```
{
  "answer_summary": "...",
  "answer_body": "..."}
}
```

Figure 8: Prompt template for Class 2

and explicitly provide a Maven dependency coordinate (groupId, artifactId, version) for the target library. For **Class 2** (Code Recommendation), the prompt provides a target safe version of the library and asks the model to generate both an explanation and corresponding implementation code under this version constraint.

The model is required to preserve the core solution approach while adapting the answer to the given safe version.

Skill Effectiveness. Figure 9 provides the secure skills for the two mitigation settings. We construct security-aware skills for RQ3 and RQ4. Apart from security-oriented constraints, these settings use the same input fields, output schema, and task objectives as the recommendation-quality setting described above. For **Class 1**, the skill asks the LLM to prefer stable versions without known critical or high-severity vulnerabilities, avoid outdated or vulnerable versions when such risks are known, and select the version with the best security posture when multiple versions are applicable. For **Class 2**, the skill asks the LLM to use the provided safe library version when possible and keep dependency configuration and API usage consistent with that version.

Setting	Added constraints
Version recommendation	(1) Prefer the latest non-vulnerable version; (2) replace outdated or vulnerable versions; (3) choose the version with the best security posture.
Code recommendation	(1) Use the provided safe version when possible; (2) keep configuration and API usage consistent with it.

Figure 9: Prompt template for Security-aware skill addition

3.5 Evaluation Metrics

We define the following evaluation metrics to evaluate the four research questions.

- **Usefulness:** Usefulness evaluates whether an answer is usable for the question, including memorization, and hallucination.
 - *Memorization (Mem):* To mitigate potential data leakage due to memorization, we perform memorization detection as follows. Given an original answer $p_i.a$, we decompose it into textual descriptions and code snippets. We compute similarities separately: 10-gram and 15-gram Jaccard similarity for text, and LCS similarity after code normalization for code. An answer is considered memorized if any similarity exceeds a threshold h . We set $h = 0.4$ and report the memorization rate as the proportion of memorized answers.
 - *Hallucination (Hal):* Hallucination evaluates the validity of recommended Maven versions. For each recommendation, we define an indicator variable hal , where $hal = 1$ if the recommended version does not exist in the Maven repository, and 0 otherwise; For code recommendation, we check the existence by Maven repository of each extracted GAV from all Maven GAVs that appear in the generated answer, a code recommendation is regarded as hallucinated, i.e., $hal = 1$, if it contains at least one extracted GAV whose version does not exist; otherwise, $hal = 0$.
- **Security Sensitiveness:**
 - *Vulnerable Version Rate (VVR)* evaluates whether LLMs produce safe version recommendations from its internal knowledge. It is defined as the proportion of recommended versions known to be vulnerable according to $\mathcal{VDB}$. We further analyze the dominant causes, including copying explicit version cues, wrong proactive upgrades, wrong proactive downgrades, composite-dependency issues, and fixed library version.
- **Migration Capability:**
 - *Unsuccessful Migration Rate:* For a recommendation m targeting library i , we define three binary indicators: safe-version declaration adaptation (VDA), dependent-library version declaration adaptation (DLVDA), and version-aware API adaptation (VAA). Specifically, $VDA(m, i) = 1$ when recommendation code declares or uses the safe version v_i^{safe} of the target library; $DLVDA(m, i) = 1$ when the related companion or transitive dependencies are updated consistently when such updates are required; $VAA(m, i) = 1$ when the API usage in the recommendation of model m is compatible with v_i^{safe} , without using deprecated APIs or obsolete API usage patterns. Based on these indicators, we define the unmigrated recommendation indicator as:

$$UMR(m, i) = \mathbb{I}[VDA(m, i) = 0 \vee DLVDA(m, i) = 0 \vee VAA(m, i) = 0],$$

, where $UR(m, i) = 1$ means that recommendation m fails to complete the safe-version migration for library i .

For **RQ1**, we examine *Memorization* and *Hallucination* to capture the usefulness of the LLM generated answer for recommended library versions. For **RQ2**, we examine *Memorization*, and *Hallucination* to capture the usefulness of LLM-generated code recommendations. We further use *Unsuccessful Migration Rate (UMR)* to evaluate whether the generated code recommendation correctly completes safe-version migration. For **RQ3**, we adopt *Memorization*, and *Hallucination* as evaluation indicators to examine whether an LLM equipped with security-aware skills achieves improved performance. For **RQ4**, we adopt *Memorization*, *Hallucination*, and *Unsuccessful Migration Rate (UMR)* as evaluation indicators to examine whether an LLM equipped with security-aware migration skills achieves improved safe-version code migration performance.

4 VERSION RECOMMENDATION QUALITY (RQ1)

We evaluate the quality of LLM-recommended library versions from two perspectives, i.e., *usefulness* and *security sensitiveness*.

Usefulness. Figure 10 shows all models achieve high basic quality. Memorization is rare (<1%), suggesting that models do not simply reproduce training data. Version Hallucination is rare for DeepSeek-R1 and Gemini-3.0-Flash-Preview (0.80% and 0.13%), while GPT-4o exhibits a higher hallucination rate of 8.00%.

Overall, these results suggest that current LLMs are generally competent at producing relevant and basically useful answers for Java libraries version recommendation. Therefore, the key issue in RQ1 is not whether the models can answer the question at all, but whether the recommended version is vulnerable.

Security-Sensitiveness. We compute the Vulnerable Version Rate VVR_m for version recommendations. Despite strong basic performance, the security quality of recommended versions is significantly weaker. As shown in Figure 11, DeepSeek-R1 recommends vulnerable versions most frequently, with a rate of 44.50%, followed by Gemini-3.0-Flash-Preview at 38.87% and GPT-4o at 25.08%.

We further compute VVR_m within each category of the matched target library to examine whether security-sensitive recommendation quality varies across library ecosystems. The vulnerable-version recommendation rate varies significantly across library types. For DeepSeek-R1, the highest vulnerability rates are observed in Database (81.97%) and Web (78.38%) libraries, followed by JSON and Testing libraries (around 30%–35%). GPT-4o shows elevated vulnerability rates in Utility (57.45%), Database (55.74%), and Web (48.65%) libraries. Gemini-3.0-Flash-Preview exhibits a pattern closer to DeepSeek-R1: its highest vulnerable-version rates also appear in Web (74.32%) and Database (68.85%) libraries, while Testing (19.47%), JSON (25.51%), Logging (26.36%), and Utility (27.66%) show relatively lower rates.

These results suggest that LLMs are particularly weak in handling dependency-intensive ecosystems (e.g., Spring and database drivers), where version compatibility and transitive dependencies are complex. In contrast, libraries with more localized usage patterns, such as Testing, JSON, and Logging libraries, tend to exhibit relatively lower vulnerability rates for some models, although the exact pattern still varies across models.

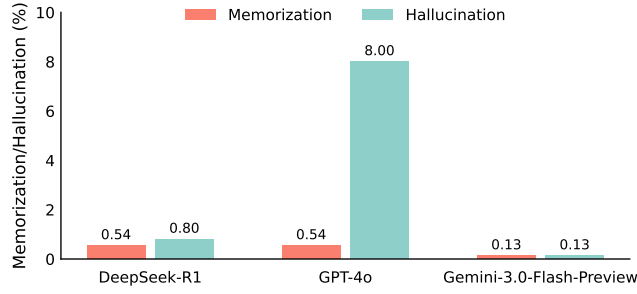


Figure 10: Usefulness of LLMs' version recommendations.

Root Cause Analysis. To understand why recommend vulnerable versions, we manually analyze the vulnerable recommendation cases identified by VRR_m and assign each case a dominant cause.

As summarized in Figure 12, the most frequent cause is copying explicit version cues from the question, which accounts for 48.5% of DeepSeek-R1 cases, 38.0% for GPT-4o, and 54.1% for Gemini-3.0-Flash-Preview. In these cases, the recommended version directly follows the version evidence appearing in the question, but the copied version is vulnerable. Another significant contributor is fixed library versions, explaining 28.9% of DeepSeek-R1 cases, 20.3% for GPT-4o, and 12.1% for Gemini-3.0-Flash-Preview. Here, the recommended version is not copied from question; instead, the model selects a commonly observed dependency version such as JUnit 4.12, Gson 2.8.6, and Spring 5.3.x, which is plausible in general Java Q&A but unsafe under the vulnerability database $\mathcal{VDB}$.

Composite dependencies are also notable, occurring in 10.5% of DeepSeek-R1, 25.1% of GPT-4o, and 25.9% of Gemini-3.0-Flash-Preview cases. These cases occur when questions involve multiple library dependencies, and the LLM selects compatible versions for all dependencies without verifying their security, thus recommending a vulnerable version for the evaluated target library. Finally, proactive version adjustments, including upgrades and downgrades, are less frequent: upgrades account for 11.4%, 14.4%, and 7.6%, while downgrades account for 0.6%, 2.1%, and 0.3% of cases for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively. We label these cases when the model changes the version evidence in question and produces a different version, but the adjusted version still vulnerable. Although these adjustments reflect attempts to resolve version conflicts or ensure API compatibility, they still often result in vulnerable recommendations.

Findings: (1) The evaluated models exhibit version hallucination rates from 0.13% to 8.00%, indicating that while LLMs handle most tasks, non-negligible hallucinations may still cause inefficiencies; (2) Although exposed to security disclosures, LLMs still frequently recommend vulnerable library versions, with rates of 44.50% for DeepSeek-R1, 38.87% for Gemini-3.0-Flash-Preview, and 25.08% for GPT-4o, highlighting insufficient security awareness; The primary causes of vulnerable version recommendations are copying explicit version cues (38%-54%) and fixed library versions (12%-29%).

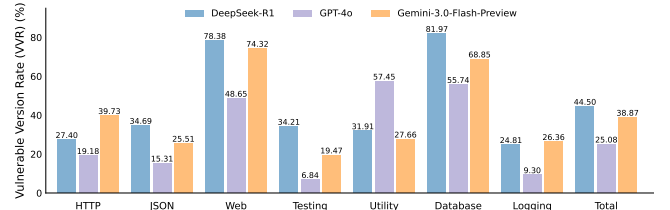


Figure 11: Vulnerable version rate by library type.

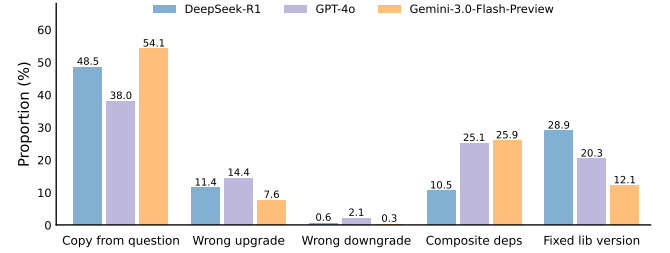


Figure 12: Causes of vulnerable-version recommendations.

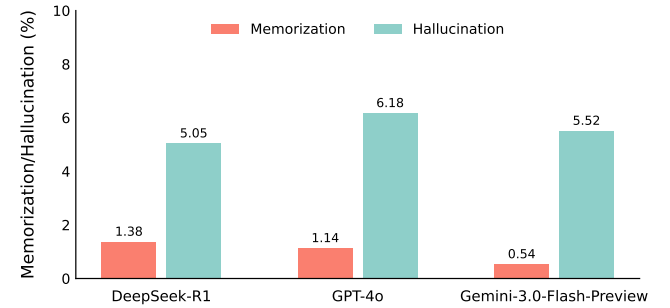


Figure 13: Usefulness of LLMs' code recommendations.

5 CODE RECOMMENDATION QUALITY (RQ2)

To evaluate the quality of LLM-recommended code under safe-version migration tasks, we investigate RQ2 from two perspectives: *usefulness* and *migration capability*.

Usefulness. We first evaluate whether the generated code recommendation is usable as an answer to the question, using Memorization, and Hallucination as defined in Section 3.5. As shown in Figure 13, Memorization remains rare across all models. DeepSeek-R1 has 1.38% (23/1672) memorized outputs, GPT-4o has 1.14% (19/1672), and Gemini-3.0-Flash-Preview has 0.54% (9/1672). This indicates that the generated code recommendations are unlikely to be simple reproductions of the original Stack Overflow answers. For hallucination, we compute the proportion of non-existent Maven GAVs among all extracted GAVs. DeepSeek-R1 produces 67 non-existent GAVs among 1327 extracted GAVs, yielding a hallucination rate of 5.05%. GPT-4o produces 6.18% (79/1279), and Gemini-3.0-Flash-Preview produces 5.52% (86/1559). Although code recommendation naturally involves more dependency mentions than version recommendation, over 93.8% of the extracted GAVs exist in the Maven repository for all models. These results suggest that the generated answers maintain high usability at the code-recommendation level.

Migration Capability. We then evaluate whether the generated recommendation completes safe-version migration. Figure 14 reports the overall UMR together with VDA, DLVDA, and VAA.

Gemini-3.0-Flash-Preview achieves the lowest UMR, with 1353/1672 (80.9%) unsuccessful migration cases. GPT-4o has a higher UMR of

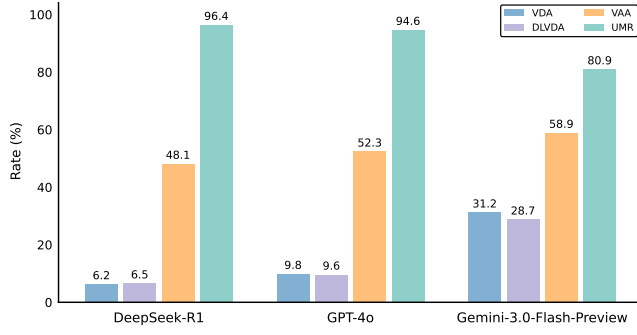


Figure 14: Migration capability of code recommendations

1582/1672 (94.6%), while DeepSeek-R1 reaches 1611/1672 (96.4%). Although Gemini-3.0-Flash-Preview performs better than the other two models, these results show that fully safe-version migration remains difficult in code recommendation tasks.

We further examine the component indicators to identify where migration failures mainly occur. For VDA, Gemini-3.0-Flash-Preview achieves the highest safe-version declaration adaptation rate, correctly declaring the target safe GAV in 521/1672 (31.2%) generated outputs. GPT-4o reaches 164/1672 (9.8%), while DeepSeek-R1 reaches 104/1672 (6.2%). For DLVDA, Gemini-3.0-Flash-Preview again performs best, with 480/1672 (28.7%) outputs providing complete and consistent dependency declarations. GPT-4o obtains 160/1672 (9.6%), and DeepSeek-R1 obtains 109/1672 (6.5%).

For VAA, Gemini-3.0-Flash-Preview also has the highest proportion of verified version-aware API adaptation, with 984/1672 (58.9%) answers using APIs compatible with the safe version. GPT-4o follows with 875/1672 (52.3%), and DeepSeek-R1 obtains 804/1672 (48.1%). The observed API migration issue rate is similar across models: 367/1672 (22.0%) for DeepSeek-R1, 371/1672 (22.2%) for GPT-4o, and 378/1672 (22.6%) for Gemini-3.0-Flash-Preview. Most API issues are partial migrations rather than complete use of outdated APIs. Specifically, old-new API mixing appears in 330/1672 (19.7%) DeepSeek-R1 outputs, 342/1672 (20.5%) GPT-4o outputs, and 356/1672 (21.3%) Gemini-3.0-Flash-Preview outputs, while fully outdated API usage appears in only 37/1672 (2.2%), 29/1672 (1.7%), and 22/1672 (1.3%) outputs, respectively. This indicates that models often attempt to migrate the code, but may still retain obsolete API calls or mix old and new usage patterns in the same answer.

We further compute the type-level $UMR_{m,t}$ to examine whether safe-version migration capability varies across library ecosystems. Figure 15 reports the $UMR_{m,t}$ values for each model and library type. Across the three models, HTTP and Testing libraries are consistently the most difficult types, all models show high UMR: DeepSeek-R1 fails in 251/253 (99.2%) cases, GPT-4o in 251/253 (99.2%), and Gemini-3.0-Flash-Preview in 246/253 (97.2%). Testing libraries exhibit a similar pattern, with UMR values of 250/253 (98.8%), 251/253 (99.2%), and 243/253 (96.0%) for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively. JSON libraries are difficult across DeepSeek-R1 and GPT-4o, with UMR values of 325/334 (97.3%) and 329/334 (98.5%), respectively. Gemini-3.0-Flash-Preview performs better but still has a high failure rate of 308/334 (92.2%). Other library types reveal clearer model-specific differences. For Web libraries, DeepSeek-R1 has a very high UMR of 547/564 (97.0%),

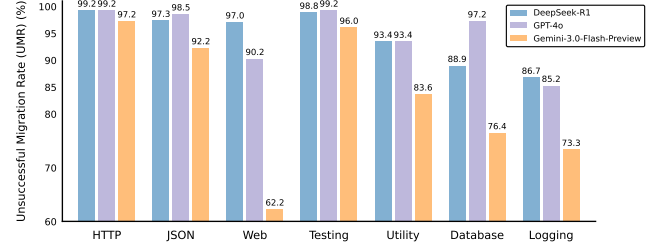


Figure 15: Migration capability categorized by library type

whereas Gemini-3.0-Flash-Preview reduces the failure rate substantially to 351/564 (62.2%). Similarly, Gemini performs relatively better on Logging and Database libraries, with UMR values of 99/135 (73.3%) and 55/72 (76.4%), respectively, while GPT-4o still fails in 70/72 (97.2%) Database cases.

Findings: We identify the following findings: (1) All evaluated models show high unsuccessful migration rates in safe-version code recommendation, reaching 96.4% for DeepSeek-R1, 94.6% for GPT-4o, and 80.9% for Gemini-3.0-Flash-Preview; (2) Dependency-level migration is the main bottleneck: VDA and DLVDA remain below 10.0% for DeepSeek-R1 and GPT-4o, and below 32.0% even for Gemini-3.0-Flash-Preview, whereas API-level adaptation is relatively higher with VAA values of 48.1%–58.9%; (3) Migration difficulty varies by library type. HTTP and Testing libraries are consistently the hardest, with UMR above 96.0% across all models, while Web, Logging, and Database libraries show stronger model-specific differences.

6 SECURITY-AWARE SKILL EFFECTIVENESS (RQ3)

We regenerate the vulnerable cases identified in RQ1 with a brief security reminder and compare the changes in basic quality metrics and security-sensitive metrics.

Usefulness Changes. The memorization rate remains low for all models, staying at 0%–0.53%. The version hallucination rate also remains low overall, ranging from 0% to 6.42% after adding skill, indicating that most versions recommended by LLMs still exist.

Security-sensitive Quality Changes. Figure 16 reports the changes in the Vulnerable Version Rate after security-aware skill. We compare pre-skill VVR_m^{base} with post-skill VVR_m^{sec} . Overall, the security-aware skill substantially reduces VVR_m . For DeepSeek-R1, 104/332 previously vulnerable cases still recommend vulnerable versions, and VVR_m decreases from $VVR_m^{base} = 44.50\%$ to $VVR_m^{sec} = 13.94\%$. GPT-4o shows a similar pattern, with 60/187 cases still vulnerable and VVR_m decreasing from $VVR_m^{base} = 25.08\%$ to $VVR_m^{sec} = 8.04\%$. Gemini-3.0-Flash-Preview benefits the most: only 20/290 cases remain vulnerable, reducing VVR_m from $VVR_m^{base} = 38.87\%$ to $VVR_m^{sec} = 2.68\%$. To further compare the effect of security-aware skill across library types, we analyze the remaining vulnerable recommendations at the library-type level. Figure 17 reports the type-level changes. The reduction ratio measures the proportion of originally vulnerable recommendations in RQ1 that are no longer vulnerable after security-aware skill adding.

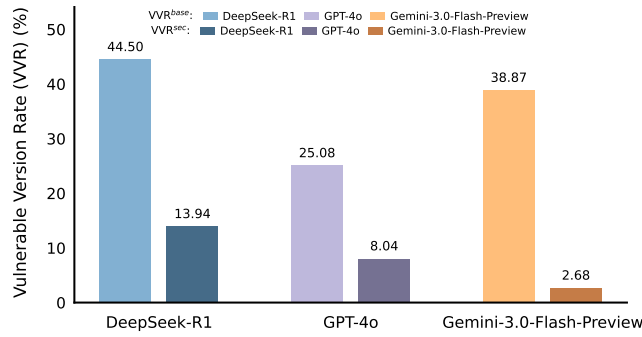


Figure 16: Comparison of Vulnerable Version Rate (VVR) Before and After Security-aware Skill

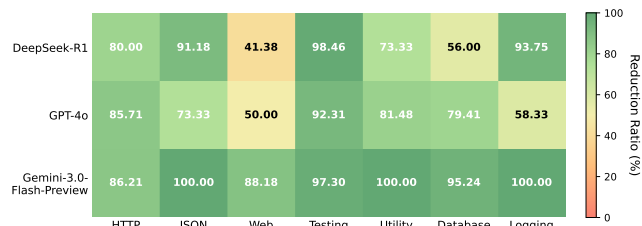


Figure 17: Vulnerable-version recommendation changes by library type after security-aware skill.

Gemini-3.0-Flash-Preview shows the most consistent type-level improvement. It removes all originally vulnerable recommendations for JSON, Utility, and Logging libraries, corresponding to 100.00% reductions for these types. It also substantially reduces Database and Testing libraries, decreasing from 42 to 2 and from 37 to 1, corresponding to reductions of 95.24% and 97.30%, respectively. Among all library types, Web libraries show the smallest reduction for Gemini-3.0-Flash-Preview, decreasing from 110 to 13, corresponding to an 88.18% reduction.

For DeepSeek-R1 and GPT-4o, Testing libraries achieve the largest reduction, decreasing from 65 to 1 for DeepSeek-R1 and from 13 to 1 for GPT-4o, corresponding to reductions of 98.46% and 92.31%, respectively. DeepSeek-R1 also largely reduces JSON and Logging libraries, with reductions of 91.18% and 93.75%, while GPT-4o substantially reduces HTTP, Utility, and Database libraries, with reductions of 85.71%, 81.48%, and 79.41%. In contrast, Web libraries show the smallest reduction for both models, decreasing from 116 to 68 for DeepSeek-R1 and from 72 to 36 for GPT-4o.

Root Cause Changes. Compared with RQ1, the number of cases annotated as copying explicit vulnerable versions is greatly reduced. For DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, such cases decrease from 161, 71, and 157 to 3, 11, and 1, respectively. Cases annotated as fixed library versions also decrease from 96, 38, and 35 to 8, 10, and 4. Among the remaining vulnerable cases, most are related to composite-dependency issues or wrong proactive upgrades, where secure version selection requires dependency-compatibility reasoning. Such cases are reduced to 93, 39, and 15 for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively.

Findings: We identify the following findings: (1) The security-aware skill keeps memorization and hallucination generally stable, from 0% to 0.53% for memorization and from 0% to 6.42% for hallucination, indicating that the skill does not cause significant quality degradation; (2) Vulnerable-version recommendations are reduced for all models, decreasing by 30.56%, 17.04%, and 36.19% for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively. (3) The reduction mainly comes from fewer copied vulnerable version cues and fixed library versions.

7 SECURITY-AWARE CODE MIGRATION QUALITY (RQ4)

We regenerate the unsuccessful migration cases identified in RQ2 with a migration-oriented security-aware skill.

Usefulness Changes. We first check whether adding security-aware skill affects the basic usability. The memorization rate remains low across all models, achieving 0.81%, 1.01%, 0.52% for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively. For hallucination, DeepSeek-R1 produces 62 non-existent GAVs among 1639 extracted GAVs after adding the skill, yielding a hallucination rate of 3.78%. GPT-4o produces 91/2232 (4.08%), and Gemini-3.0-Flash-Preview produces 35/1078 (3.25%). Compared with RQ2, the hallucination rate decreases slightly for all models, by 1.27, 2.10, and 2.27 percentage points, indicating that the skill does not harm the basic usability.

Migration Capability Changes. Figure 18 reports the changes in UMR after adding security-aware skill. Compared with RQ2, all three models show lower unsuccessful migration rates. For DeepSeek-R1, the number of unsuccessful migration cases decreases from 1611 to 1517, reducing UMR from 96.4% to 90.7%. GPT-4o shows the largest absolute reduction, decreasing from 1582 to 1396 unsuccessful cases, with UMR decreasing from 94.6% to 83.5%. Gemini-3.0-Flash-Preview decreases from 1353 to 1188 unsuccessful cases, reducing UMR from 80.9% to 71.1%. Thus, the skill reduces UMR by 5.6, 11.1, and 9.9 percentage points for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively.

We further examine the three component indicators of UMR to identify where the reduction comes from. For VDA, DeepSeek-R1 increases from 104/1672 (6.2%) to 230/1672 (13.8%), GPT-4o from 164/1672 (9.8%) to 419/1672 (25.1%), and Gemini-3.0-Flash-Preview from 521/1672 (31.2%) to 691/1672 (41.3%). These correspond to improvements of 7.5, 15.3, and 10.2 percentage points, respectively. For DLVDA, DeepSeek-R1 increases from 109/1672 (6.5%) to 217/1672 (13.0%), GPT-4o from 160/1672 (9.6%) to 388/1672 (23.2%), and Gemini-3.0-Flash-Preview from 480/1672 (28.7%) to 637/1672 (38.1%). The corresponding improvements are 6.5, 13.6, and 9.4 percentage points. All three models improve in VDA and DLVDA after adding the skill, indicating that the skill helps models provide more complete and consistent dependency declarations. For VAA, the improvement is smaller. DeepSeek-R1 increases from 804/1672 (48.1%) to 936/1672 (56.0%), GPT-4o from 875/1672 (52.3%) to 1035/1672 (61.9%), and Gemini-3.0-Flash-Preview from 984/1672 (58.9%) to 1085/1672 (64.9%). The corresponding improvements are 7.9, 9.6, and 6.0 percentage points.

We also analyze the UMR reduction across library types. Figure 19 shows the largest reductions in Web libraries: DeepSeek-R1

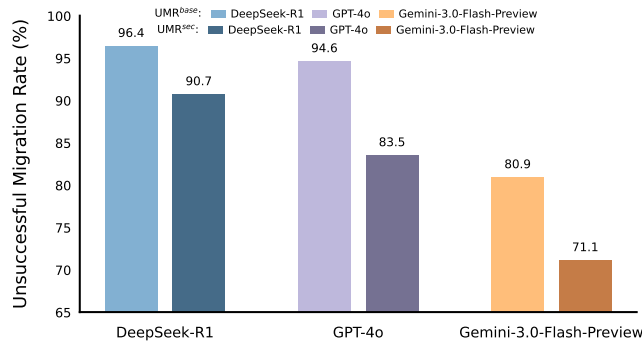


Figure 18: Comparison of Unsuccessful Migration Rate (UMR) before and after security-aware skill.

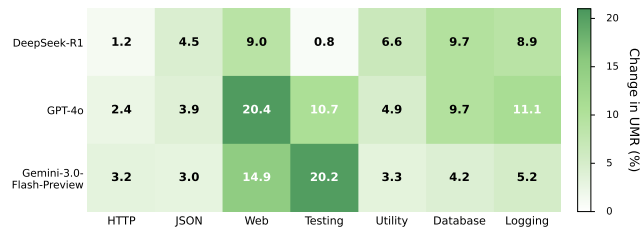


Figure 19: Unsuccessful Migration Rate (UMR) changes by library type after security-aware skill.

decreases from 547/564 (97.0%) to 496/564 (87.9%), GPT-4o from 509/564 (90.2%) to 394/564 (69.9%), and Gemini-3.0-Flash-Preview from 351/564 (62.2%) to 267/564 (47.3%), i.e., by 9.0, 20.4, and 14.9 percentage points. Testing libraries also show clear reductions for GPT-4o and Gemini-3.0-Flash-Preview, by 10.7 and 20.2 percentage points, while DeepSeek-R1 changes little (0.8 points). In contrast, HTTP and JSON libraries remain difficult, with only limited reductions (1.2–3.2% for HTTP and 3.0–4.5% for JSON); post-skill $UMR_{m,t}$ still remains at 98.0%, 96.8%, and 94.1% for HTTP and 92.8%, 94.6%, and 89.2% for JSON.

Findings: We identify the following findings: (1) The security-aware skill reduces unsuccessful migration for all models. Compared with RQ2, UMR decreases by 5.6%, 11.1%, and 9.9% for DeepSeek-R1, GPT-4o, and Gemini-3.0-Flash-Preview, respectively. (2) The main improvement comes from dependency-level migration. Safe-version declaration adaptation increase by +126, +255, and +170 cases, and dependent-library version declaration adaptation increase by +108, +228, and +157 cases. (3) The skill is most effective for Web libraries code migration, with decreases of 9.0%, 20.4%, and 14.9% across the three models. However, HTTP and JSON libraries remain hard to migrate, with post-skill $UMR_{m,t}$ still above 98.0%, 96.8%, and 94.1% for HTTP and 92.8%, 94.6%, and 89.2% for JSON.

8 DISCUSSION

8.1 Threats to Validity

First, our study relies on posts collected from Stack Overflow, which may not fully cover the diversity of real-world development scenarios. Developers in industry may encounter contexts, constraints, or proprietary environments that are underrepresented in such public

forums. However, Stack Overflow remains one of the largest and most widely used platforms for developer knowledge sharing, and prior studies have demonstrated its representativeness for common programming tasks. Second, we focus on a set of 21 popular Java libraries, which may limit the generalizability of our findings. However, these libraries are widely adopted and cover diverse functionalities. Our results may not extend to other libraries. Third, we use the number of correctly adapted APIs to measure migration capability. While this metric captures whether the generated code aligns with the target library version at the API level, it may not fully reflect other important aspects such as semantic correctness, runtime behavior, or security robustness.

8.2 Implications

For developers: Our study indicates that useful answer does not guarantee secure or migration-correct library recommendations. The three models still recommend vulnerable versions in 44.50%, 25.08%, and 38.87% of cases, and fail safe-version migration in 96.4%, 94.6%, and 80.9% of code-recommendation cases. Developers should avoid blindly trusting LLM responses, especially when they involve library selection or version migration. Instead, they should carefully verify the recommended versions and validate the generated code against official documentation and security advisories.

For researchers: Current LLMs still show weak security-aware reasoning. Although security-aware skill reduces vulnerable version by 30.56, 17.04, and 36.19 percentage points, post-skill unsuccessful migration rates remain high at 90.7%, 83.5%, and 71.1%. It highlights the need for more advanced techniques to improve LLMs' security-aware capabilities, particularly in handling library versioning and vulnerability mitigation. This includes better training data curation, reasoning mechanisms, and evaluation benchmarks.

For LLM developers: We find that the main limitations are weak built-in security awareness and insufficient support for dependency-consistent migration. In particular, dependency-level adaptation remains limited, with VDA at only 6.2%, 9.8%, and 31.2%, and DLVDA at 6.5%, 9.6%, and 28.7%. LLMs should be equipped with stronger internal mechanisms to ensure the safety of their outputs, such as incorporating vulnerability knowledge bases, enforcing secure version constraints, and applying post-generation validation. Strengthening these mechanisms is critical to reducing the risk of recommending vulnerable or outdated dependencies.

9 CONCLUSIONS

We conducted an empirical study on the reliability of LLM-based library recommendation, focusing on both version selection and associated code usage. Overall, our study reveals that LLM-based library recommendation systems are not yet sufficiently reliable for security-critical software development tasks. While lightweight prompting can provide partial improvements, it does not fundamentally resolve the underlying issues of hallucination and weak security awareness. A robust mechanisms to integrate security knowledge into LLM reasoning is needed for future research.

REFERENCES

- [1] Dibyendu Brinto Bose. 2025. From prompts to properties: Rethinking llm code generation with property-based testing. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1660–1665.
- [2] Keyuan Cheng, Xudong Shen, Yihao Yang, Tengyue Wang, Yang Cao, Muhammad Asif Ali, Hanbin Wang, Lijie Hu, and Di Wang. 2025. Codemenv: Benchmarking large language models on code migration. In *Findings of the Association for Computational Linguistics: ACL 2025*. 2719–2744.
- [3] Tristan Coignon, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering*. 79–89.
- [4] Xing Cui, Jingzheng Wu, Xiang Ling, and Tianyue Luo. 2025. We Know What You're Looking For: Recommendation for Large-Scale Open Source Software. In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 277–288.
- [5] Leuson Da Silva, Jordan Samhi, and Foutse Khomh. 2025. LLMs and Stack Overflow discussions: Reliability, impact, and challenges. *Journal of Systems and Software* 230 (2025), 112541.
- [6] DeepSeek. 2025. DeepSeek-R1. <https://www.deepseek.com/>.
- [7] Mingzhe Du, Luu Anh Tuan, Bin Ji, Xiaobao Wu, Yuhao Qing, Dong Huang, Terry Yue Zhuo, Qian Liu, and See Kiong Ng. 2025. Codearena: A collective evaluation platform for llm code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. 502–512.
- [8] Duncan. 2016. Jackson ObjectMapper setSerializationInclusion() not working. <https://stackoverflow.com/questions/37406817/jackson-objectmapper-setserializationinclusion-not-working>.
- [9] Ekin Can Erkuş, Cansen Çağlayan Yılmaz, and Mustafa Umut Demirezen. 2026. ArkTS code generation: A comprehensive evaluation with large language models. *Empirical Software Engineering* 31, 4 (2026), 109.
- [10] GitHub. [n. d.]. GitHub Advisory Database. <https://github.com/advisories>. Accessed: 2025-12-01.
- [11] Google. 2025. Gemini 3.0 Flash Preview. <https://deepmind.google/technologies/gemini/>.
- [12] Google BigQuery. [n. d.]. Stack Overflow Dataset. <https://console.cloud.google.com/marketplace/product/stack-exchange/stack-overflow>. Accessed: 2025-11-26.
- [13] Donghee Han, Hwanjun Song, and Mun Yong Yi. 2025. Rethinking LLM-Based Recommendations: A Personalized Query-Driven Parallel Integration. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 8395–8419.
- [14] Hao He, Runzhi He, Haiqiao Gu, and Minghui Zhou. 2021. A large-scale empirical study on Java library migrations: prevalence, trends, and rationales. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 478–490.
- [15] Hao He, Yulin Xu, Xiao Cheng, Guangtai Liang, and Minghui Zhou. 2021. MigrationAdvisor: Recommending library migrations from large-scale open-source data. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 9–12.
- [16] Dong Huang, Jie M. Zhang, Qingwen Bu, Xiaofei Xie, Junjie Chen, and Heming Cui. 2025. Bias testing and mitigation in llm-based code generation. *ACM Transactions on Software Engineering and Methodology* 35, 1 (2025), 1–31.
- [17] Jiani Huang, Shijie Wang, Liangbo Ning, Wenqi Fan, Shuaiqiang Wang, Dawei Yin, and Qing Li. 2026. Towards next-generation recommender systems: A benchmark for personalized recommendation assistant with llms. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*. 217–226.
- [18] Md Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. 2025. Using llms for library migration. *arXiv preprint arXiv 2504* (2025).
- [19] Chumeng Jiang, Jiayin Wang, Weizhi Ma, Charles LA Clarke, Shuai Wang, Chuhan Wu, and Min Zhang. 2025. Beyond utility: Evaluating llm as recommender. In *Proceedings of the ACM on Web Conference 2025*. 3850–3862.
- [20] Sathvik Joel, Jie Wu, and Fatemeh Fard. 2024. A survey on llm-based code generation for low-resource and domain-specific programming languages. *ACM Transactions on Software Engineering and Methodology* (2024).
- [21] Samia Kabir, David N Udo-Imeh, Bonan Kou, and Tianyi Zhang. 2024. Is stack overflow obsolete? an empirical study of the characteristics of chatgpt answers to stack overflow questions. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–17.
- [22] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? An empirical study on the impact of security advisories on library migration. *Empirical Software Engineering* 23, 1 (2018), 384–417.
- [23] Kaibei Li, Jie Zou, Qika Lin, Weikang Guo, Qinyang He, and Yang Yang. 2026. ScotRec: Social Chain-of-Thought LLM Reasoning for Recommendation. In *Proceedings of the ACM Web Conference 2026*. 6242–6251.
- [24] Shuang Li, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He, and Weiye Shang. 2024. Assessing the performance of ai-generated code: A case study on github copilot. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 216–227.
- [25] Shuang Li, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He, and Weiye Shang. 2026. Performance analysis of AI-generated code: A case study of Copilot, Copilot Chat, CodeLlama, and DeepSeek-Coder models. *Empirical Software Engineering* 31, 3 (2026), 62.
- [26] Fake Lin, Binbin Hu, Zhi Zheng, Xi Zhu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, and Tong Xu. 2026. Token-level Collaborative Alignment for LLM-based Generative Recommendation. *arXiv preprint arXiv:2601.18457* (2026).
- [27] Lin Ling, Fazle Rabbi, Song Wang, and Jinqiu Yang. 2025. Bias unveiled: Investigating social bias in LLM-generated code. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 39. 27491–27499.
- [28] Fang Liu, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. *IEEE Transactions on Software Engineering* (2026).
- [29] Qijiong Liu, Lu Fan, and Xiao-Ming Wu. 2025. Legommenders: A comprehensive content-based recommendation library with llm support. In *Companion Proceedings of the ACM on Web Conference 2025*. 753–756.
- [30] Qidong Liu, Xian Wu, Yejing Wang, Zijian Zhang, Feng Tian, Yefeng Zheng, and Xiangyu Zhao. 2024. Llm-esr: Large language models enhancement for long-tailed sequential recommendation. *Advances in Neural Information Processing Systems* 37 (2024), 26701–26727.
- [31] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-driven development and llm-based code generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1583–1594.
- [32] MvnRepository. 2025. Popular Maven Artifacts. <https://mvnrepository.com/popular>.
- [33] National Vulnerability Database. [n. d.]. NVD - National Vulnerability Database. <https://nvd.nist.gov/>. Accessed: 2024-12-01.
- [34] Open Source Vulnerabilities. [n. d.]. OSV. <https://osv.dev/>. Accessed: 2024-12-01.
- [35] OpenAI. 2024. GPT-4o. <https://openai.com/>.
- [36] Gergely Orosz. 2026. *Stack overflow is almost dead*. Retrieved April 20, 2026 from <https://blog.practicengineer.com/stack-overflow-is-almost-dead/>
- [37] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [38] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do llms consider security? an empirical study on responses to programming questions. *Empirical Software Engineering* 30, 4 (2025), 101.
- [39] Alfred Santa Molison, Marcia Moraes, Glauca Melo, Fabio Santos, and Wesley KG Assuncao. 2025. Is LLM-Generated Code More Maintainable & Reliable Than Human-Written Code?. In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 151–162.
- [40] Nidhish Shah, Zulkuf Genc, and Dogu Araci. 2024. Stackeval: Benchmarking llms in coding assistance. *Advances in Neural Information Processing Systems* 37 (2024), 36976–36994.
- [41] Joseph Spracklen, Raveen Wijewickrama, AHM Nazmus Sakib, Anindya Maiti, and Bimal Viswanath. 2025. We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*. 3687–3706.
- [42] Stack Overflow. 2026. Stack Overflow. <https://stackoverflow.com/>.
- [43] Stack Overflow user. 2018. Maven dependencies conflict selenium-firefox-driver vs tint-runner. <https://stackoverflow.com/questions/52850482/maven-dependencies-conflict-selenium-firefox-driver-vs-tint-runner>.
- [44] Cédric Teyton, Jean-Rémy Fallier, Marc Palyart, and Xavier Blanc. 2014. A study of library migrations in java. *Journal of Software: Evolution and Process* 26, 11 (2014), 1030–1052.
- [45] Bohao Wang, Jiawei Chen, Feng Liu, Changwang Zhang, Jun Wang, Canghong Jin, Chun Chen, and Can Wang. 2026. Does LLM Focus on the Right Words? Mitigating Context Bias in LLM-based Recommenders. *arXiv preprint arXiv:2510.10978* (2026).
- [46] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Llmrec: Large language models with graph augmentation for recommendation. In *Proceedings of the 17th ACM international conference on web search and data mining*. 806–815.
- [47] Qian Zhang, Tsz-On Li, Ying Wang, Li Li, and Shing-Chi Cheung. 2025. Cross2OH: Enabling Seamless Porting of C/C++ Software Libraries to OpenHarmony. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1744–1755.
- [48] Yifan Zhang, Tsong Yueh Chen, Rubing Huang, Matthew Pike, Dave Towey, Zhihao Ying, and Zhi Quan Zhou. 2025. Comparative analysis of styles in LLM-generated code for LeetCode problems: A preliminary study. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1625–1630.
- [49] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjuan Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical

- code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.
- [50] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [51] Bo Zhou, Jiaqi Shi, Ying Wang, Li Li, Tsz On Li, Hai Yu, and Zhiliang Zhu. 2025. Porting software libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1445–1466.
- [52] Bingzhe Zhou, Xinying Wang, Shengbin Xu, Yuan Yao, Minxue Pan, Feng Xu, and Xiaoxing Ma. 2023. Hybrid api migration: A marriage of small api mapping models and large language models. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware*. 12–21.